

Live Build Session — Full Stack Product Engineering

Candidate Instructions — please read before your interview

What this is

You'll spend the session building a small full-stack application **live, with us in the room** — and we expect much of that building to happen through your prompts, so how you direct your AI tooling is as much a part of the craft as the code it produces. We've sent you a starter repo (the `shell` folder) containing a Docker environment. It does exactly one thing: serves a **Hello World!** page at `http://localhost:3000`. Everything else is up to you.

Use (or add) **any AI tooling you like** — Claude Code, Cursor, Codex, Copilot, anything. We want to see how you **work with stakeholders to translate real-world problems into technical solutions** and **use modern tools to organize solutions in an appropriate manner** — not how well you type syntax from memory or build complex algorithms.

Before the session — checklist

- ☐ Install Docker and confirm it runs on your machine.
- ☐ Unzip the starter repo. From its root, run `make build` then `make start`.
- ☐ Open `http://localhost:3000` and confirm you see **Hello World!**
- ☐ Set up your editor and your AI tooling of choice so it's ready to go.
- ☐ Skim the two files in `prompts/` — you'll paste them into your AI agent at the start and end of the session.
- ☐ Read the brief below and come with questions.

Do this ahead of time. Interview minutes spent debugging Docker are minutes you don't get back.

The environment

- The starter "app" is a plain Node HTTP server (`workspace/server/src/index.js`) serving Hello World! on port 3000 — that's the entire codebase.
- The container has **Node.js 22**, git, curl, and standard build tools.
- `./workspace` on your machine is mounted at `/workspace` in the container — edit on your host, run inside the container.

- Ports **3000**, **5173**, and **8000** are published for dev servers and APIs.
- VS Code users: open the folder and use the Dev Containers extension ("*Reopen in Container*"), or simply edit on your host and use `make shell`.

Command	What it does
<code>make build</code>	Build the container image
<code>make start</code>	Start the container (prints Hello World!)
<code>make shell</code>	Bash shell inside the container
<code>make logs</code>	Follow container logs
<code>make stop</code>	Stop the container

Stack

- **Backend:** Node.js
- **Frontend:** React with Tailwind CSS — keep it simple.

That's the default we provide, but it's not a requirement: **you may switch to any stack you want ahead of the session** — swap the Dockerfile, the runtime, the starter server, whatever suits you. Just have it done *before* the live session (stack setup is not what the hour is for), and make sure `make build && make start` still serves Hello World! at `http://localhost:3000`.

The situation

You're meeting with the two people behind an independent recording studio. Today, everything they do — bookings, schedules, client notes — lives on pen and paper. They've decided they want custom software.

You'll meet the **owner**, who wants the business to make more money, and the **front-desk person**, who runs the day-to-day.

In this room, **you are the technical expert** — we are the business users. We know our studio inside out; we don't know what software can or should do for it. That part is your job.

Build them something.

That's the whole brief, and that's deliberate. The details live with the stakeholders — **come prepared to ask questions**, at the start and throughout the session.

How the hour goes

Time	Phase	What happens
0–5	Setup check	Show us Hello World! at <code>localhost:3000</code> , then paste <code>prompts/1-start-logging.md</code> into your AI agent as its first message.
5–15	Discovery	Interview us. We'll answer as the stakeholders — the owner and the front-desk person.
15–50	Build	Build, narrating out loud the whole way. You can keep asking us questions at any point.
50–60	Demo & wrap-up	Demo what you built as if you're showing it to the stakeholders. Paste <code>prompts/2-stop-logging.md</code> into your agent and send us the resulting <code>PROMPTS.md</code> .

What we expect in each phase

- **Setup.** This is pre-work, not part of the hour. Arrive with the container already verified, your editor open, and your AI agent ready to go. The five minutes at the top are for showing us it runs — not for installing Docker.
- **Requirements gathering.** Talk to *both* stakeholders — they see the business differently, and neither has the whole picture. Before you write any code, we expect to hear you say out loud what you're going to build, and just as importantly, what you're deliberately leaving out. If we haven't agreed on that, keep asking.
- **Building.** Working software over ambitious scaffolds — we'd rather see one thing running end-to-end than five things half-built. Write it like code someone else will maintain after you leave, keep narrating, and keep asking questions when you hit a decision the conversation hasn't covered. It's fine — good, even — to cut scope midway when the hour demands it; tell us what you're cutting and why.

Ground rules

- **Narrate out loud, the entire time.** What you're thinking, what you're asking your AI tools to do, what you're checking, and why. Silence is the only wrong answer.
- **Treat us as your stakeholders.** Ask us anything about the business, the users, and the priorities. We'll answer in character.
- **AI tools are encouraged.** Drive them however you normally do.

- **Prompt logging.** Your first message to your AI agent will be `prompts/1-start-logging.md` and your last will be `prompts/2-stop-logging.md`. Together they produce a `PROMPTS.md` log of everything you asked your agent — you'll send it to us after the session.
- **Recording.** The meeting will be video recorded so we can review it afterward.
- **Working software beats ambitious software.** The session is one hour. Scope like it.

One free hint: this studio needs less software than you think. The hard part is finding out which part it actually needs — and that answer isn't in this document. Build less, ask more.